Operating System Security

Week 2: Access Control

Motivation for Discussing Access Control

Recall that a successful OS should ensure the security of all processes

- i.e., it should *control access* to resources, such that one process cannot (maliciously or not) impact the execution of another.

Access Control

- An operating system provides an access enforcement mechanism.

AEM

- An *access enforcement mechanism* authorizes requests (e.g., system calls)
  - …from multiple *subjects* (e.g. users, processes, etc.)
  - …to perform *operations* (e.g., read, write, etc.) on *objects* (e.g., files, sockets, etc.).

- Most of today's material is from Chapter 2 of textbook:
  [*] T. Jaegar, "Operating System Security", Morgan & Claypool Publishers, 2008.

Access Control

- Two fundamental concepts of access control:
  - a protection system that defines the access control specification and
  - a reference monitor that is the system's access enforcement mechanism that enforces this specification.

Protection system

- A protection system consists of a protection state, which describes:
  - the operations that system subjects can perform on system objects, and
  - a set of protection state operations, which enable modification of that state.

- One way of viewing a protection state is through an *Access Matrix*

	File 1	File 2	File 3
Process 1	Read	Read, Write	Read, Write
Process 2	-	Read	Read, Write

Figure 2.1: Lampson's Access Matrix

Access Control Lists (ACLs)

- Often used instead of Access Matrices.
  - Reason: more efficient use of space, as matrices are sparse.

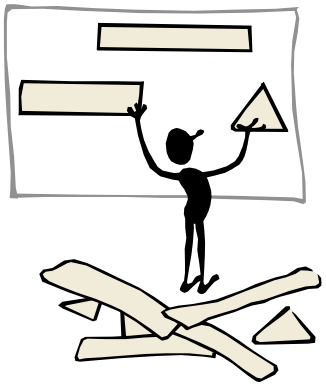
- Essentially object columns of an Access Matrix, describing which subjects have access to the object.

- Most systems use ACLs as they are intuitive for administering permissions to an object.

Discretionary Access Control Systems

These systems leave the protection state at the discretion of users and any untrusted processes they execute

- Ideally, this is not what we want



We want to ensure:

- A particular protection state
- That all possible protection states derivable from it will not provide unauthorized access
- Ensuring this is known as the **safety problem**

Example

- Assume File 1 stores a secret value, e.g., a document containing access codes to a facility
- Assume Process 1 is malicious
  - Process 1 could leak the access codes by changing Process 2's permissions to File1!
  - To ensure the secrecy of File 1, all processes with access to it must not be able to leak it through its permissions.

	File 1	File 2	File 3
Process 1	Read	Read, Write	Read, Write
Process 2	-	Read	Read, Write

Figure 2.1: Lampson's Access Matrix

Addressing the Safety Problem

- The protection state must be defined **based on the secrecy and integrity** of user data and processes
  - Objects: based on the amount of secrecy needed, or sensitivity of the data
  - Subjects: based on the amount of "clearance" they have, corresponding with how well they are trusted

- Additionally, no untrusted processes may be allowed to perform protection state operations
  - e.g., giving permissions to another process that does not have the required "clearance level"

- This leads us to ***Mandatory protection systems***

Mandatory Protection Systems

- Protection systems that can only be modified by trusted administrators via trusted software
- Subjects and objects are represented by labels

Mandatory access control

- Mandatory protection systems are also called *mandatory access control* (MAC) systems because the protection system is immutable to untrusted processes.

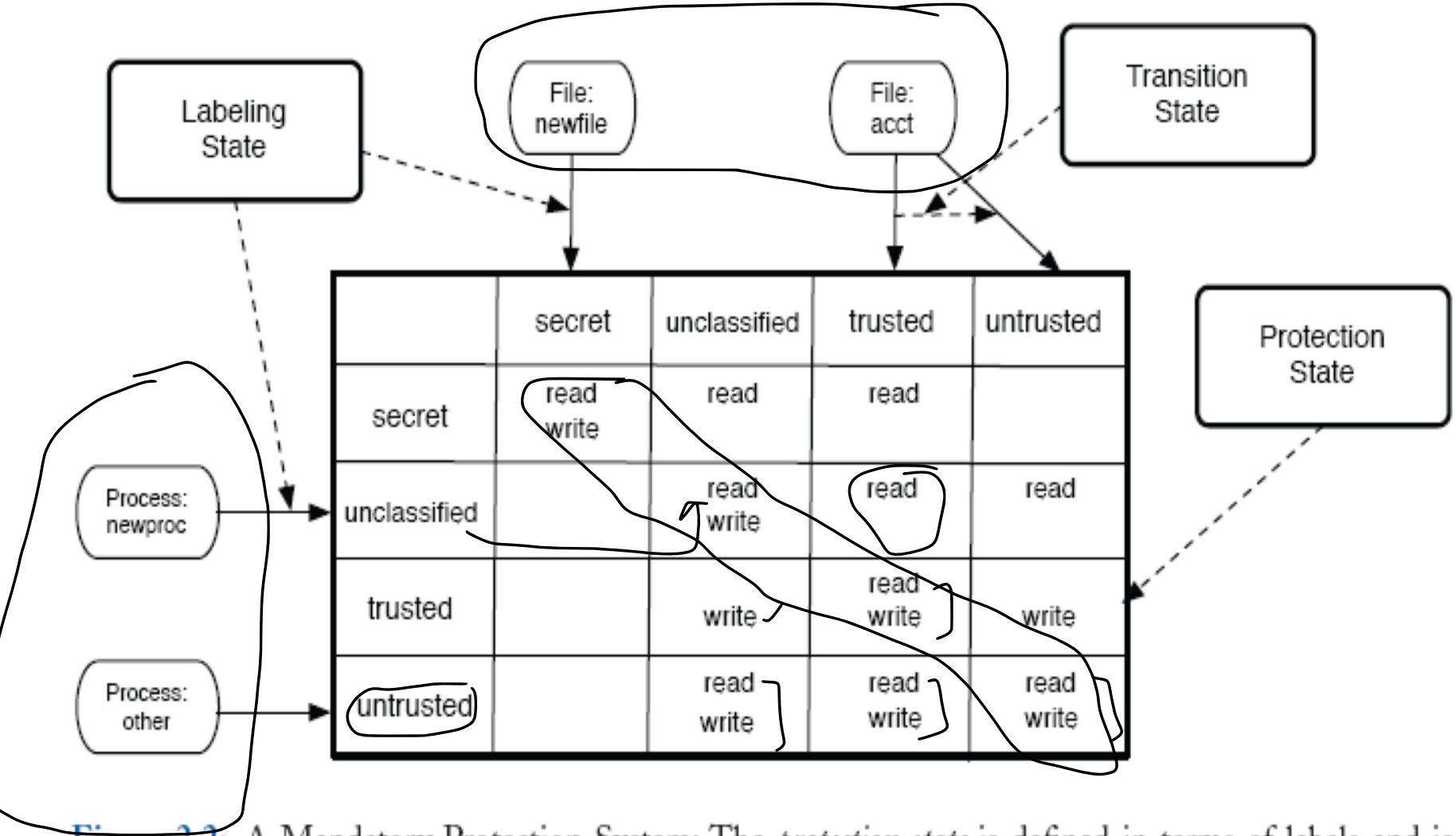


Figure 2.2: A Mandatory Protection System: The *protection state* is defined in terms of labels and is immutable. The immutable *labeling state* and *transition state* enable the definition and management of labels for system subjects and objects.

Mandatory Protection Systems

- Verbal description of the diagram on previous slide:

- Consists of the following state representations
  - A *mandatory protection state*: describes the operations that subject labels may take upon object labels.
  - A *labeling state*: maps processes and resource objects to labels
  - A *transition state*: describes the legal ways that processes and resource objects may be relabeled. Defines how labels change based on modifications (e.g., an untrusted process modifies a file)

Reference Monitor

- The classical access enforcement mechanism.
- It takes a request as input, and returns a binary response indicating whether the request is authorized by the reference monitor's access control policy.

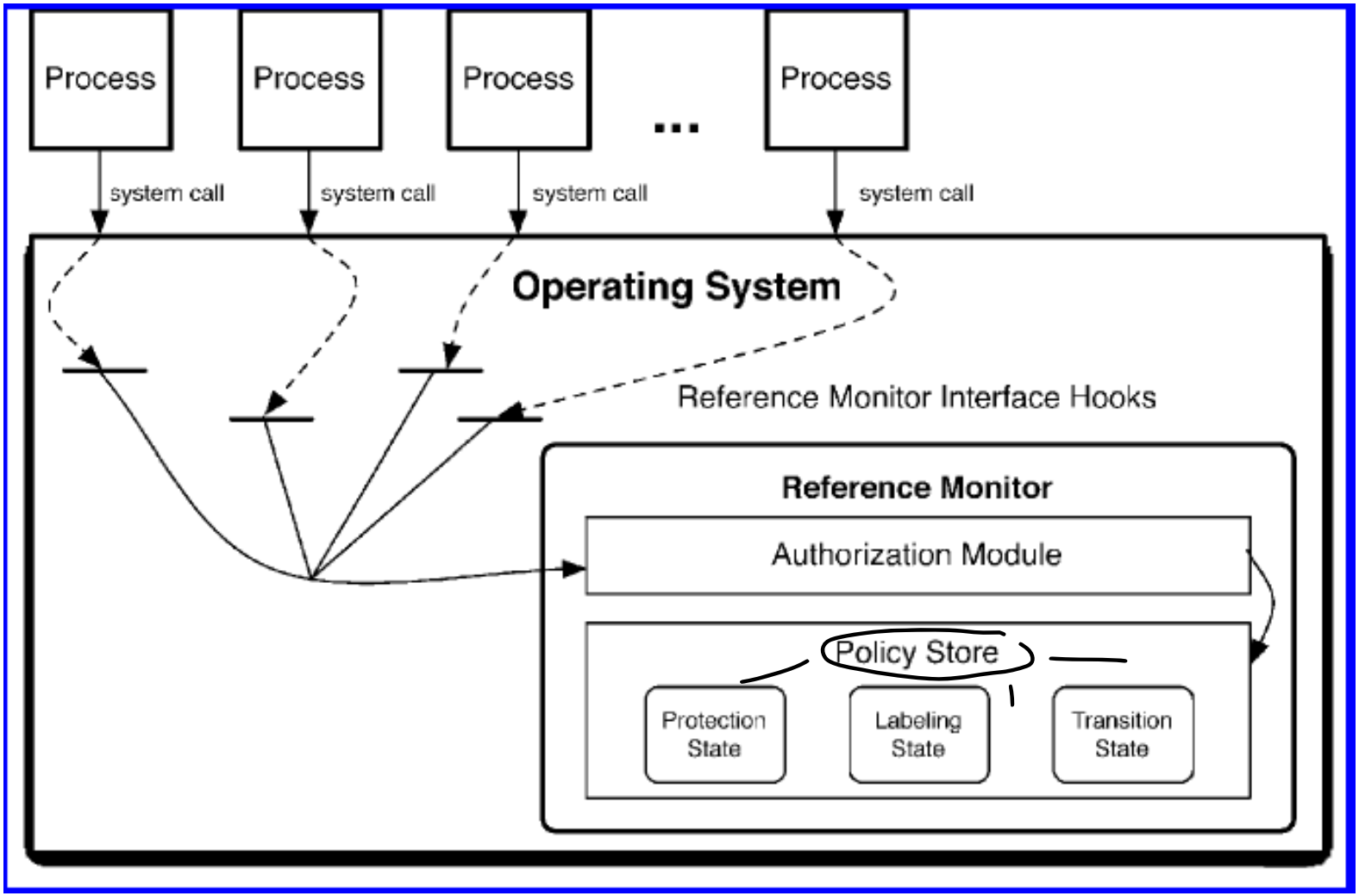


Figure 2.3: A *reference monitor* is a component that authorizes access requests at the *reference monitor interface* defined by individual *hooks* that invoke the reference monitor's *authorization module* to submit an authorization query to the *policy store*. The policy store answers authorization queries, labeling queries, and label transition queries using the corresponding states.

Reference Monitor Interface

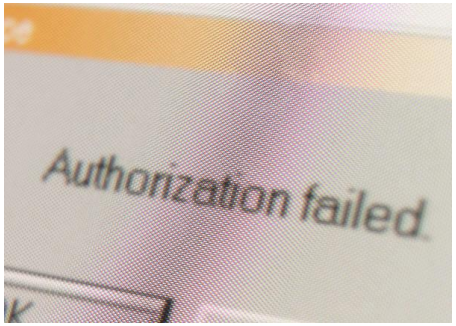
It defines where the authorization module needs to be invoked.

It ensures that all security-sensitive operations are authorized by the access enforcement mechanism.

It determines where access enforcement is necessary, and the information the reference monitor needs to authorize that request.

Reference Monitor Authorization Module

The core of the reference monitor.



Takes the interface's inputs (e.g., process id, object references, and system call names) and converts these to a query for the reference monitor's policy store.

The challenge for the authorization module is to map actual process IDs to subject labels, and object references to object labels.

Reference Monitor Policy Store

The database for the protection state, labeling state, and transition state.

An authorization query from the authorization module is answered by the policy store.

- Queries are of the form {subject_label, object_label, operation_set} and return a binary authorization reply.

Also accepts labeling queries, which set new labels, and transition queries, which change the labels of the resources.

Limitations: an important example

Assume that File2 stores a high integrity value (e.g., a public key)

- Process 2 could exploit a vulnerability in Process 1 (e.g., a buffer overflow).
- This means that Process 2 has injected code it wants to run into Process 1 – and the system will see its actions as actions by Process 1.
- Process 2's injected code could then use Process 1's permissions to modify it to be an attackers' public key (for which he/she has the private key).

	File 1	File 2	File 3
Process 1	Read	Read, Write	Read, Write
Process 2	-	Read	Read, Write

Figure 2.1: Lampson's Access Matrix

Limitations of Protection Systems

Stop and think about what the mandatory protection system is capable of achieving:

- Good: it would prevent Process 2's injected code from changing the protection state
  - Because Process 1 is not allowed to change the protection state.
- Sad: it cannot prevent other code, if injected into Process 1, from accessing objects that Process 1 has permissions for.

Recap

Core concepts discussed in this module:

- Access control
- Protection systems
- Reference monitor
- Discretionary protection systems
- Mandatory protection systems